先选对安装包

0.2.2 / Windows x64、Mac 苹果芯片、Mac Intel / 2026-10-14

朋友看到什么?

你发安装包链接，朋友在浏览器里下载文件；浏览器可能直接存入“下载”，也可能询问保存位置。你发整个 Release 页面，朋友先看到版本说明和 Assets 附件列表，再点对应安装包。它不是打开一个在线工作日志网站。

Mac 分两个芯片版本

让朋友打开苹果菜单 → 关于本机。显示“芯片 Apple M…”选择 macos-arm64.dmg；显示“处理器 Intel…”选择 macos-x64.dmg。目标系统为 macOS 13.5 或更新版本，不按每一代系统分别做包。下载后双击 DMG，把Work Log.app 拖到 Applications，再从应用程序启动。

Windows 目前提供一个版本

Intel / AMD 的 64 位 Windows 电脑选择 windows-x64-setup.exe。兼容目标为 Windows 10 / 11；当前在 Windows 11 完成测试，Windows 10 尚待单独实测。不提供 32 位 x86 或 Windows ARM 原生包。双击后有安装向导和安装位置，安装后从桌面打开。

当前真实状态

Windows x64、Mac 苹果芯片版与 Intel 版均已构建，并通过原生保存、主题和 PDF 导出测试。Mac 在 macOS 15 验证，未逐代系统实测。

安装包尚未完成发布者签名及 Apple 公证。Mac 系统可能阻止首次打开；只对确认来源的文件参照 Apple 官方说明处理，不关闭系统保护。

上传公开源码

账号 Benjamin-can-fly / 仓库 work-journal / MIT 开源

1. 使用工作目录中的干净源码

打开工作目录下的“release-prep/Work-Log-0.2.2-Release/01-Source”。这里已剔除私人记录、附件、运行缓存和本机打包工具。不要上传原项目中的 data，也不要把整个桌面文件夹传上去。

2. 推荐 GitHub Desktop

安装 desktop.github.com 提供的官方客户端并登录自己的账号。选择 File → Add local repository，选上述源码文件夹。若不是仓库，点击 create a repository，确保创建位置仍是这个源码文件夹，而不是新建的空子文件夹。

检查 Changes，填写 Summary“准备 0.2.2 桌面预览版”，点击 Commit to main。源码包已有 README、MIT LICENSE、截图和三个目标的构建流程。不要添加安装包目录。

3. Publish repository

仓库名填写 work-journal，取消 Keep this code private，发布到 Benjamin-can-fly。网址将是 github.com/Benjamin-can-fly/work-journal。如改仓库名，README 中的下载、Issues 链接也要一起改。

备用入口

源码目录中的“Publish-to-GitHub.cmd”可以引导登录并上传公开仓库，但需要网络可访问 GitHub 设备登录服务。当前项目已经公开发布，后续更新可继续使用已配置的 Git 登录。不要把密码或验证码发到聊天里。图形客户端登录更容易检查每一步。

上传后先检查

网页上应看见 README、src、server、desktop、scripts、tests、.github。不得出现 data/journal.json、私人附件或 .env。MIT 已按你的选择放入 LICENSE。

生成三个同版本安装包

上传源码后，通过 GitHub 的 Windows 和 Mac 电脑分别构建

1. 手动启动 Actions

仓库网页 → Actions → Build desktop previews → Run workflow → 选择 main → Run workflow。如果要求启用 Actions，先启用。不要先发布没有安装包的正式版本。

2. 等待三个任务成功

Windows、macOS arm64、macOS x64 必须分别显示成功。Mac 任务会编译原生窗口，实际保存记录并导出 PDF，验证通过才打包。Windows 也检查桌面保存和导出。红色失败时打开日志，修复后重新运行，不能把失败当作已适配。

3. 下载构建附件

在运行页面底部 Artifacts 下载三个 installer 压缩包。解压后应得到一个 windows-x64-setup.exe、两个架构的 .dmg，以及 .sha256。文件名版本都应是 0.2.2。不要重命名旧安装包冒充新版。

4. 再做实际使用检查

首次 Mac 构建成功后，请朋友检查打开、中文输入、图片拖入、保存、关闭重开和 PDF 导出。Mac 构建机测试不能代表所有系统和硬件。名称与图标已统一为 Work Log；代码签名仍待完成。

5. 发布预览版 Release

仓库 → Releases → Draft a new release。创建标签 v0.2.3，标题填写新版本号，需要试用验证时勾选 pre-release。附加三个经过验证的安装包、校验文件和使用说明 PDF，再发布。Actions 不会自动替你对外发布 Release。

Source code (zip) 是 GitHub 提供的源码。让朋友下载手动附加的 DMG 或 EXE。当前没有自动更新；新版需下载覆盖安装，数据保留，更新前建议导出 JSON。

通过微信发给朋友

只发对应安装包或下载链接，不需要发整个项目

最方便的消息

“这是工作日志安装包。你在苹果菜单 → 关于本机看一下芯片：M 系列下载 arm64，Intel 下载 x64。下载 DMG 后打开，把软件拖到应用程序里，再打开。”

发送链接

发布后在 Release 的 Assets 中复制对应 DMG 链接发到微信。公开 Release 通常不要求朋友登录 GitHub。微信内置页面不能下载时复制到 Safari；网络受限时，你先下载 DMG，再用电脑微信发送文件。

直接发送文件

将正确的 DMG 拖入微信聊天即可。不要发 Windows EXE 给 Mac，也不要发源码 ZIP。微信可能限制文件传输；遇到限制可用可访问的网盘分享同一安装包。

数据与隐私

你的个人数据不在安装包中。朋友首次打开是空白本机空间。Windows 默认在 %LOCALAPPDATA%\WorkJournal\data；Mac 默认在 ~/Library/Application Support/WorkJournal/data。没有在线账号同步。

先别发给朋友的东西

不要发送 Node.js、.build-tools、node_modules、完整源码、个人 JSON 备份，或没有真正编译成功的 Mac 文件。分享记录时应单独导出所需 PDF，并检查正文和附件名称是否涉及隐私。

README 已排好，怎么继续改

README.md 是 GitHub 仓库首页；图片使用公开示例数据

现在的页面结构

顶部名称与一句用途 → 主界面截图 → 功能表 → 三种下载选择 → 折叠的其他界面截图 → 快捷操作 → 数据位置 → 开发方式 → MIT。先让普通用户知道怎么用，再给开发者看构建说明。

改名字和介绍

打开 README.md，替换顶部名称和一句简介。避免堆装饰标签、空下载链接和没有证据的“全平台完美支持”。确定图标后，在标题上方加入 logo 图片。

换截图

四张截图在 docs/images，名称是 journal.png、tasks.png、timeline.png、dashboard.png。替换后 Markdown 中相对路径不用改。使用示例记录、隐藏私人内容，保持一致窗口尺寸。

预览和提交

GitHub 网页打开 README.md → 编辑 → Preview，确认表格、链接、图片正常后提交。或在本地编辑器预览后，使用 GitHub Desktop 的 Commit 和 Push origin。

后续需要更新的地方

Mac 构建和实测通过后，把状态更新为实际结果；发布新安装包后更新版本说明。下载地址只指向真实 Release。增加功能时同时补使用说明和测试。

官方参考

GitHub Desktop 上传项目

创建 Releases

Apple 首次打开说明